大纲

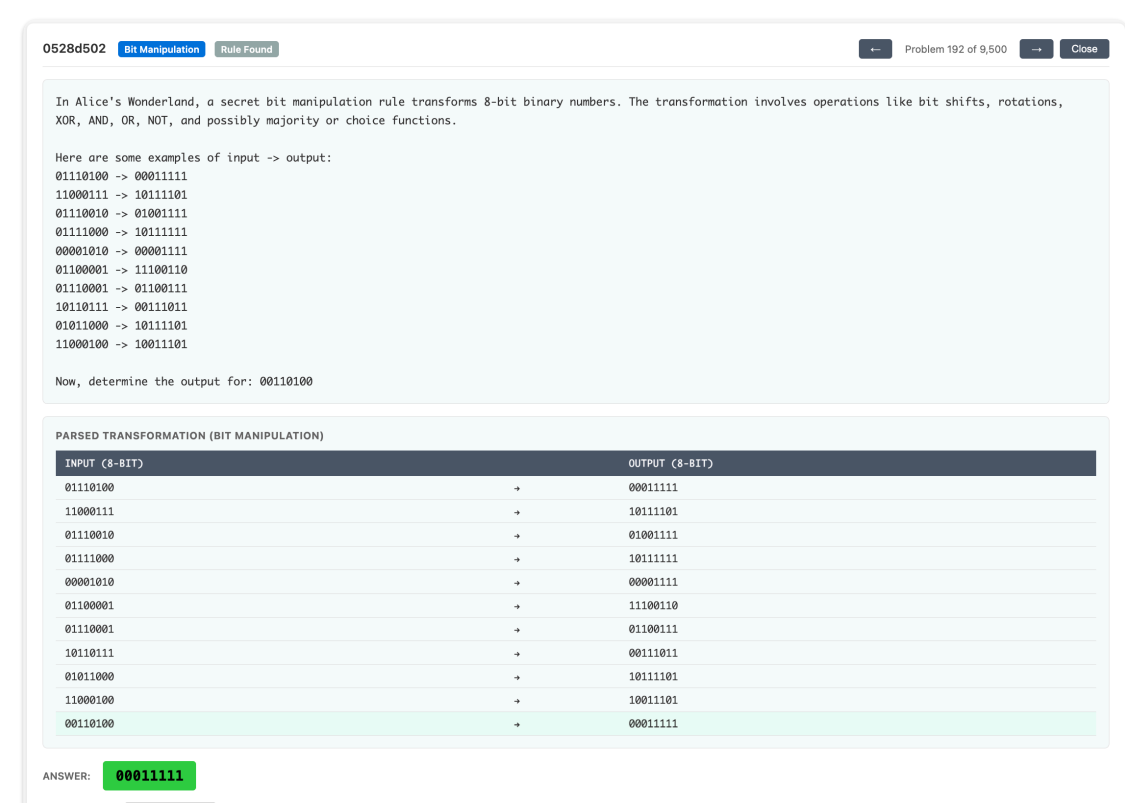
1° 赛题回顾

2° 主流解法讲解

3° Top 方案分享

4° 经验总结

赛题回顾



0528d502 Bit Manipulation Rule Found ← Problem 192 of 9,500 → Close

```
In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary numbers. The transformation involves operations like bit shifts, rotations,
XOR, AND, OR, NOT, and possibly majority or choice functions.

Here are some examples of input -> output:
01110100 -> 00011111
11000111 -> 10111101
01110010 -> 01001111
01111000 -> 10111111
00001010 -> 00001111
01100001 -> 11100110
01110001 -> 01100111
10110111 -> 00111011
01011000 -> 10111101
11000100 -> 10011101

Now, determine the output for: 00110100
```

PARSED TRANSFORMATION (BIT MANIPULATION)

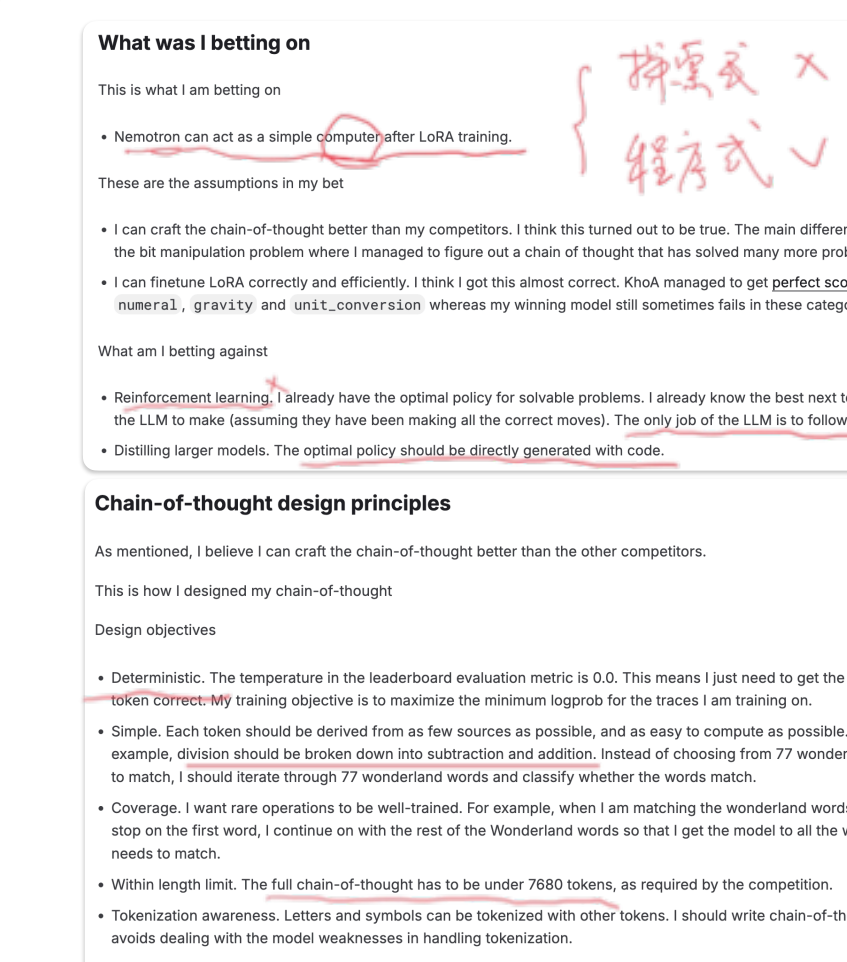
INPUT (8-BIT)		OUTPUT (8-BIT)
01110100	→	00011111
11000111	→	10111101
01110010	→	01001111
01111000	→	10111111
00001010	→	00001111
01100001	→	11100110
01110001	→	01100111
10110111	→	00111011
01011000	→	10111101
11000100	→	10011101
00110100	→	00011111

ANSWER: 00011111

SUBMISSION: 00011111

Synthetic
nemotron.huikang.dev

主流方案



What was I betting on

This is what I am betting on

- Nemotron can act as a simple computer after LoRA training.

These are the assumptions in my bet

- I can craft the chain-of-thought better than my competitors. I think this turned out to be true. The main differentiator was the bit manipulation problem where I managed to figure out a chain of thought that has solved many more problems.
- I can finetune LoRA correctly and efficiently. I think I got this almost correct. KhoA managed to get perfect scores on `numeral`, `gravity` and `unit_conversion` whereas my winning model still sometimes fails in these categories.

What am I betting against

- Reinforcement learning. I already have the optimal policy for solvable problems. I already know the best next token for the LLM to make (assuming they have been making all the correct moves). The only job of the LLM is to follow the policy.
- Distilling larger models. The optimal policy should be directly generated with code.

Chain-of-thought design principles

As mentioned, I believe I can craft the chain-of-thought better than the other competitors.

This is how I designed my chain-of-thought

Design objectives

- Deterministic. The temperature in the leaderboard evaluation metric is 0.0. This means I just need to get the most likely token correct. My training objective is to maximize the minimum logprob for the traces I am training on.
- Simple. Each token should be derived from as few sources as possible, and as easy to compute as possible. For example, division should be broken down into subtraction and addition. Instead of choosing from 77 wonderland words to match, I should iterate through 77 wonderland words and classify whether the words match.
- Coverage. I want rare operations to be well-trained. For example, when I am matching the wonderland words, I do not stop on the first word, I continue on with the rest of the Wonderland words so that I get the model to all the words it needs to match.
- Within length limit. The full chain-of-thought has to be under 7680 tokens, as required by the competition.
- Tokenization awareness. Letters and symbols can be tokenized with other tokens. I should write chain-of-thought that avoids dealing with the model weaknesses in handling tokenization.
- Generalizable. Apart from certain information (Wonderland words for Cipher), I should not be training the model to memorize answers. I should be aware of signs where the model is merely memorizing answers in the training set.

{ 拼凑式 ×
程序式 ✓

围绕程序式解题

↓

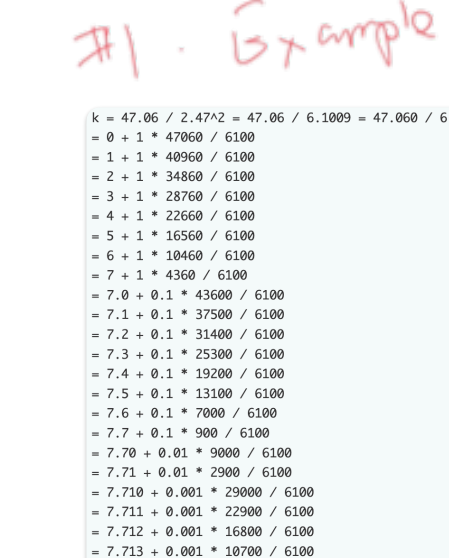
把解题过程转化成 CoT 路径

↓

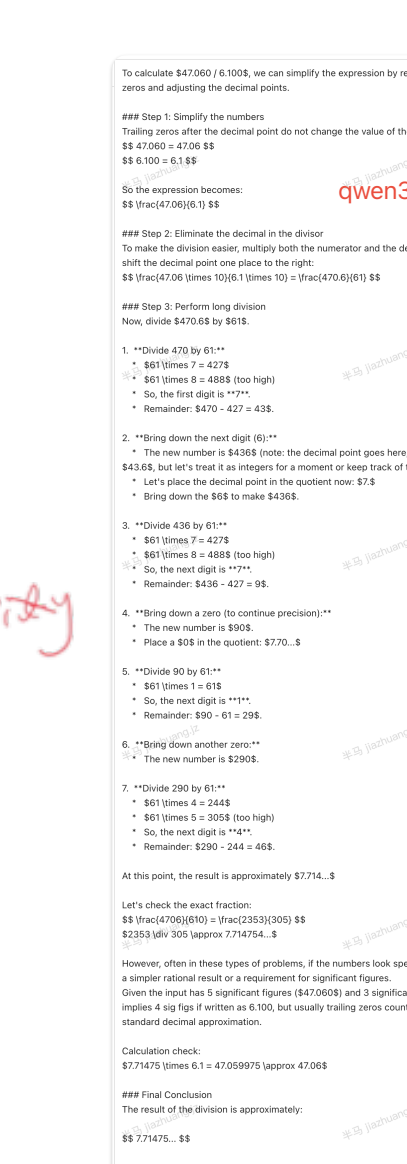
通过SFT让模型学会解题路径

(把LLM当计算器用)

#1. Example of division



```
k = 47.06 / 2.47^2 = 47.06 / 6.1009 = 47.060 / 6.100
= 0 + 1 * 47060 / 6100
= 1 + 1 * 40960 / 6100
= 2 + 1 * 34860 / 6100
= 3 + 1 * 28760 / 6100
= 4 + 1 * 22660 / 6100
= 5 + 1 * 16560 / 6100
= 6 + 1 * 10460 / 6100
= 7 + 1 * 4360 / 6100
= 7.0 + 0.1 * 43600 / 6100
= 7.1 + 0.1 * 37500 / 6100
= 7.2 + 0.1 * 31400 / 6100
= 7.3 + 0.1 * 25300 / 6100
= 7.4 + 0.1 * 19200 / 6100
= 7.5 + 0.1 * 13100 / 6100
= 7.6 + 0.1 * 7000 / 6100
= 7.7 + 0.1 * 900 / 6100
= 7.70 + 0.01 * 9000 / 6100
= 7.71 + 0.01 * 2900 / 6100
= 7.710 + 0.001 * 29000 / 6100
= 7.711 + 0.001 * 22900 / 6100
= 7.712 + 0.001 * 16800 / 6100
= 7.713 + 0.001 * 10700 / 6100
= 7.714 + 0.001 * 4600 / 6100
= 7.714
```



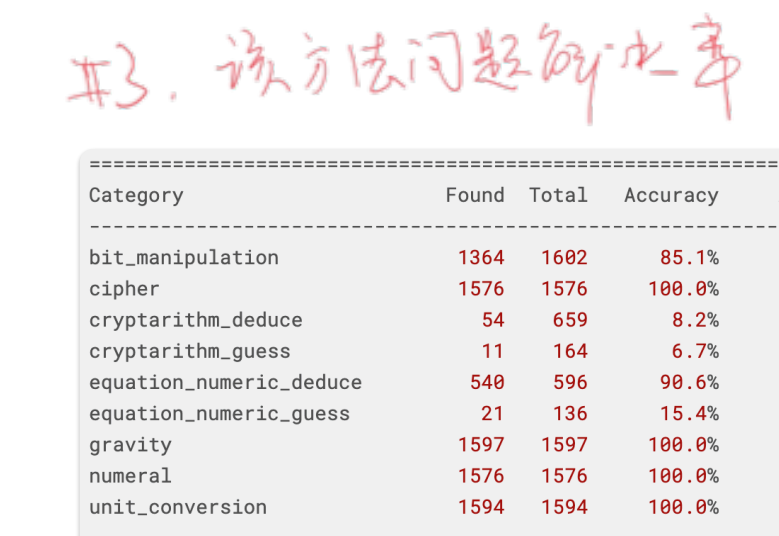
qwen3.7-plus

```
To calculate $47.060 / 6.100$, we can simplify the expression by removing the trailing zeros and adjusting the decimal points.

### Step 1: Simplify the numbers
Trailing zeros after the decimal point do not change the value of the number:
$$ 47.060 = 47.06 $$
$$ 6.100 = 6.1 $$

So the expression becomes:
$$ \frac{47.06}{6.1} $$

### Step 2: Eliminate the decimal in the divisor
To make the division easier, multiply both the numerator and the denominator by 10 to shift the decimal point one place to the right:
$$ \frac{47.06 \times 10}{6.1 \times 10} = \frac{470.6}{61} $$

### Step 3: Perform long division
Now, divide $470.6$ by $61$.

1. **Divide 470 by 61:**
   * $61 \times 7 = 427$
   * $61 \times 8 = 488$ (too high)
   * So, the first digit is **7**.
   * Remainder: $470 - 427 = 43$.

2. **Bring down the next digit (6):**
   * The new number is $436$ (note: the decimal point goes here, so it's actually $43.6$, but let's treat it as integers for a moment or keep track of the decimal).
   * Let's place the decimal point in the quotient now: $7.$
   * Bring down the $6$ to make $436$.

3. **Divide 436 by 61:**
   * $61 \times 7 = 427$
   * $61 \times 8 = 488$ (too high)
   * So, the next digit is **7**.
   * Remainder: $436 - 427 = 9$.

4. **Bring down a zero (to continue precision):**
   * The new number is $90$.
   * Place a $0$ in the quotient: $7.70...$

5. **Divide 90 by 61:**
   * $61 \times 1 = 61$
   * So, the next digit is **1**.
   * Remainder: $90 - 61 = 29$.

6. **Bring down another zero:**
   * The new number is $290$.

7. **Divide 290 by 61:**
   * $61 \times 4 = 244$
   * $61 \times 5 = 305$ (too high)
   * So, the next digit is **4**.
   * Remainder: $290 - 244 = 46$.

At this point, the result is approximately $7.714...$

Let's check the exact fraction:
$$ \frac{4706}{610} = \frac{2353}{305} $$
$2353 \div 305 \approx 7.714754...$

However, often in these types of problems, if the numbers look specific, there might be a simpler rational result or a requirement for significant figures.
Given the input has 5 significant figures ($47.060$) and 3 significant figures ($6.100$ implies 4 sig figs if written as 6.100, but usually trailing zeros count), let's provide the standard decimal approximation.

Calculation check:
$7.71475 \times 6.1 = 47.059975 \approx 47.06$

### Final Conclusion
The result of the division is approximately:

$$ 7.71475... $$

Rounded to two decimal places:
**7.71**

Rounded to three decimal places:
**7.715**
```

#2. Example of Gravity

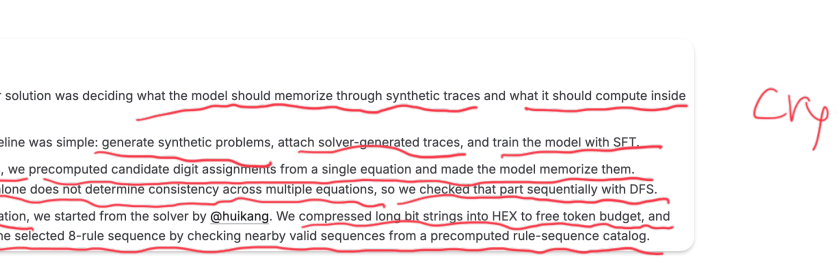
Synthetic
nemotron.huikang.dev

#3. 该方法问题解决率.



```
==========================================================================
Category                      Found   Total   Accuracy    Avg ms
--------------------------------------------------------------------------
bit_manipulation               1364    1602      85.1%       0.8
cipher                         1576    1576     100.0%       0.1
cryptarithm_deduce               54     659       8.2%       0.0
cryptarithm_guess                11     164       6.7%       0.0
equation_numeric_deduce         540     596      90.6%       0.6
equation_numeric_guess           21     136      15.4%       0.6
gravity                        1597    1597     100.0%       0.1
numeral                        1576    1576     100.0%       0.0
unit_conversion                1594    1594     100.0%       0.1
--------------------------------------------------------------------------
TOTAL                          8333    9500      87.7%       0.2
==========================================================================
```

Top1 方案

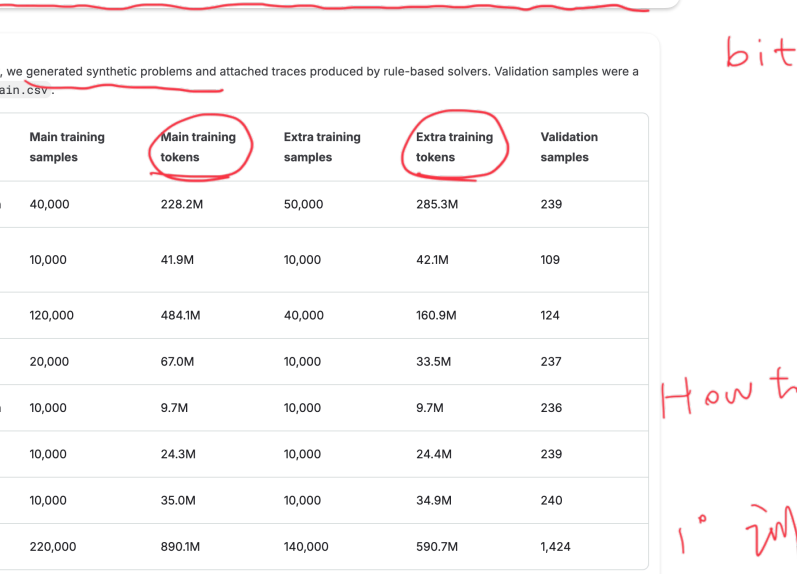


Summary

- The core of our solution was deciding what the model should memorize through synthetic traces and what it should compute inside the trace.
- The overall pipeline was simple: generate synthetic problems, attach solver-generated traces, and train the model with SFT.
- For cryptarithm, we precomputed candidate digit assignments from a single equation and made the model memorize them. Memorization alone does not determine consistency across multiple equations, so we checked that part sequentially with DFS.
- For bit manipulation, we started from the solver by @huikang. We compressed long bit strings into HEX to free token budget, and then repaired the selected 8-rule sequence by checking nearby valid sequences from a precomputed rule-sequence catalog.

Cryptarithm { memorization / DFS

bit-manipulation { HEX / recheck



Synthetic Data

For each category, we generated synthetic problems and attached traces produced by rule-based solvers. Validation samples were a 15% split from `train.csv`.

Category	Main training samples	Main training tokens	Extra training samples	Extra training tokens	Validation samples
bit manipulation	40,000	228.2M	50,000	285.3M	239
equation numeric	10,000	41.9M	10,000	42.1M	109
cryptarithm	120,000	484.1M	40,000	160.9M	124
text cipher	20,000	67.0M	10,000	33.5M	237
numeral system	10,000	9.7M	10,000	9.7M	236
unit conversion	10,000	24.3M	10,000	24.4M	239
gravity	10,000	35.0M	10,000	34.9M	240
total	220,000	890.1M	140,000	590.7M	1,424

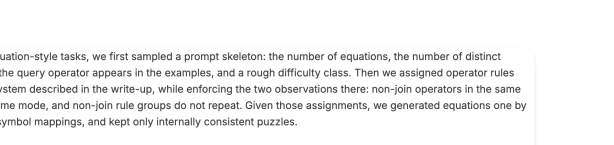
After the main training finished, there was still time before the competition ended, and bit manipulation still had room for improvement. Therefore, we continued training on extra data with a higher proportion of bit manipulation samples. The extra training token counts in the table are for the full extra data pool. Because of time constraints, extra training was run for 6,500 steps. With effective batch size 16, the actual number of processed samples was `104,000`.

How to generate synthetic?

1° 训练数据集分布分析

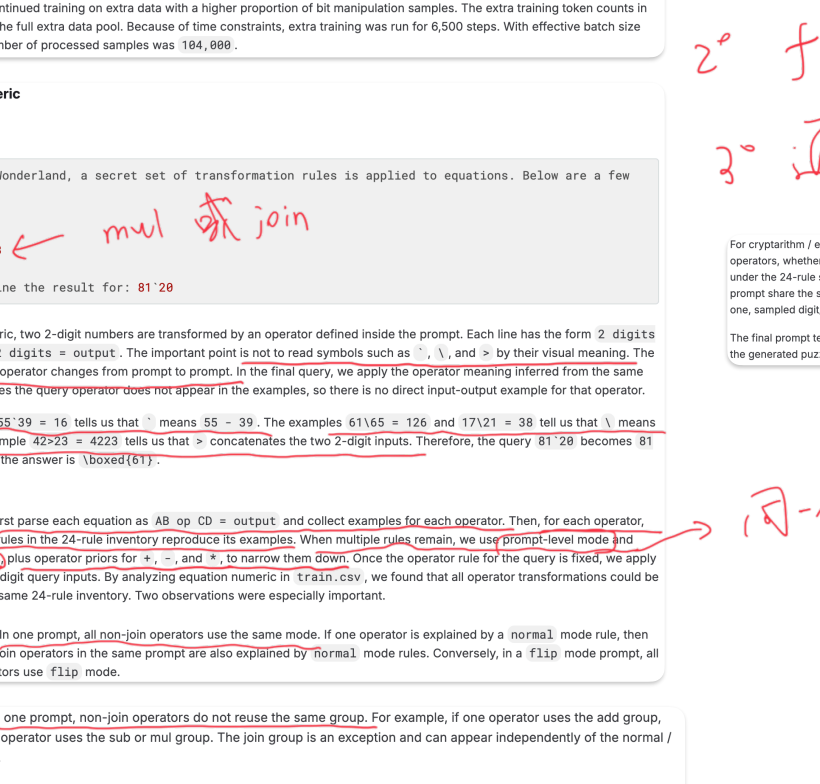
2° f: x → y (逆向工程)

3° 通过"分布 + f"采样数据



For cryptarithm / equation-style tasks, we first sampled a prompt skeleton: the number of equations, the number of distinct operators, whether the query operator appears in the examples, and a rough difficulty class. Then we assigned operator rules under the 24-rule system described in the write-up, while enforcing the two observations there: non-join operators in the same prompt share the same mode, and non-join rule groups do not repeat. Given those assignments, we generated equations one by one, sampled digit/symbol mappings, and kept only internally consistent puzzles.

The final prompt text was rendered from templates matching the competition format. In a separate step, we ran our rule solver on the generated puzzles and rendered the solver trace.



Equation Numeric

Task Format

```
In Alice's Wonderland, a secret set of transformation rules is applied to equations. Below are a few
examples:
55`39 = 16
61\65 = 126
42>23 = 4223
17\21 = 38
Now, determine the result for: 81`20
```

mul 或 join

In equation numeric, two 2-digit numbers are transformed by an operator defined inside the prompt. Each line has the form `2 digits + operator + 2 digits = output`. The important point is not to read symbols such as `` ` ``, `\`, and `>` by their visual meaning. The meaning of each operator changes from prompt to prompt. In the final query, we apply the operator meaning inferred from the same prompt. Sometimes the query operator does not appear in the examples, so there is no direct input-output example for that operator.

In this example, ``55`39 = 16`` tells us that `` ` `` means `55 - 39`. The examples `61\65 = 126` and `17\21 = 38` tell us that `\` means addition. The example `42>23 = 4223` tells us that `>` concatenates the two 2-digit inputs. Therefore, the query ``81`20`` becomes `81 - 20 = 61`, and the answer is `\boxed{61}`.

Solution

In the trace, we first parse each equation as `AB op CD = output` and collect examples for each operator. Then, for each operator, we check which rules in the 24-rule inventory reproduce its examples. When multiple rules remain, we use prompt-level mode and group constraints, plus operator priors for `+`, `-`, and `*`, to narrow them down. Once the operator rule for the query is fixed, we apply that rule to the 2-digit query inputs. By analyzing equation numeric in `train.csv`, we found that all operator transformations could be explained by the same 24-rule inventory. Two observations were especially important.

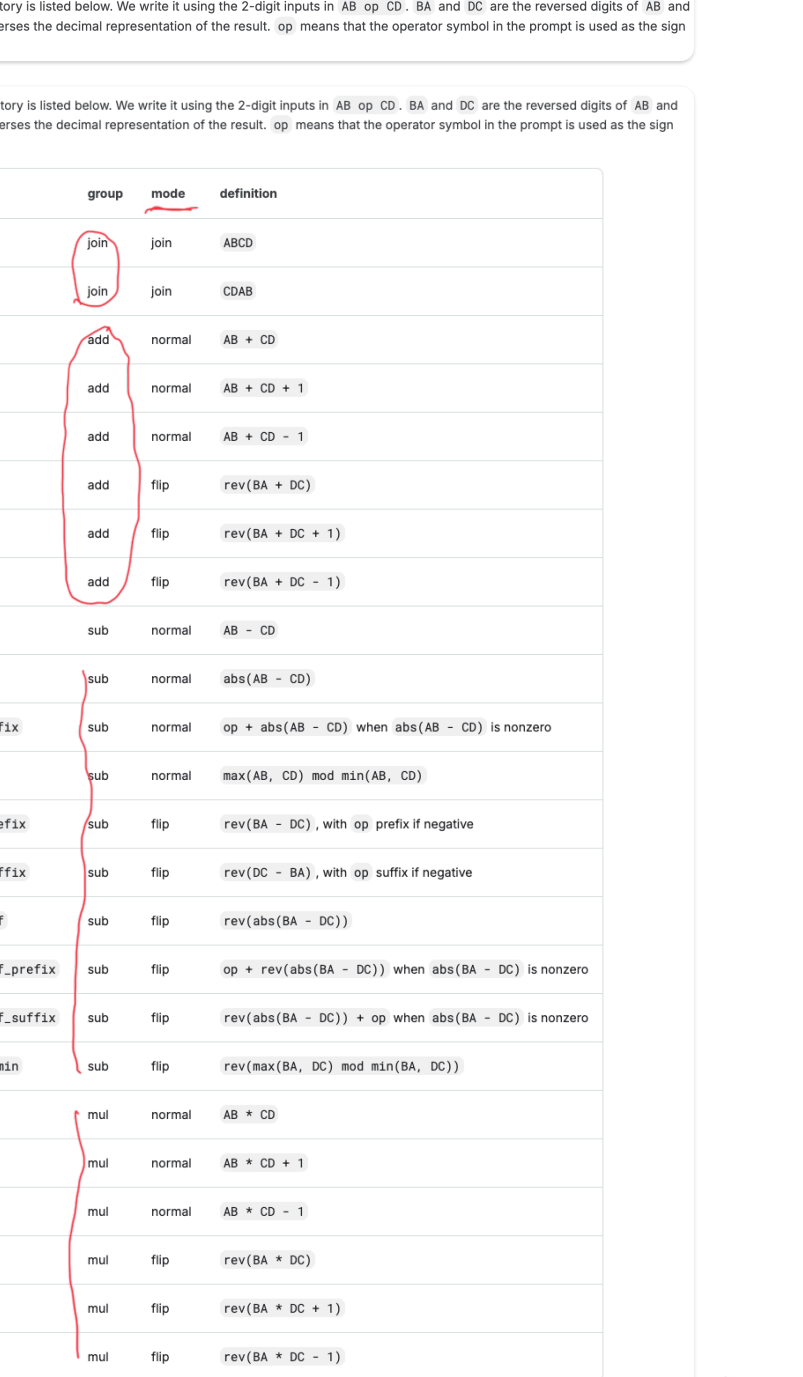
- Observation 1: In one prompt, all non-join operators use the same mode. If one operator is explained by a `normal` mode rule, then the other non-join operators in the same prompt are also explained by `normal` mode rules. Conversely, in a `flip` mode prompt, all non-join operators use `flip` mode.

不同符号属于不同 group

同一个 prompt 中所有 Eq 都采用相同的 reverse 方式 flip

- Observation 2: In one prompt, non-join operators do not reuse the same group. For example, if one operator uses the add group, another non-join operator uses the sub or mul group. The join group is an exception and can appear independently of the normal / flip mode choice.

The 24-rule inventory is listed below. We write it using the 2-digit inputs in `AB op CD`. `BA` and `DC` are the reversed digits of `AB` and `CD`. `rev(x)` reverses the decimal representation of the result. `op` means that the operator symbol in the prompt is used as the sign marker.

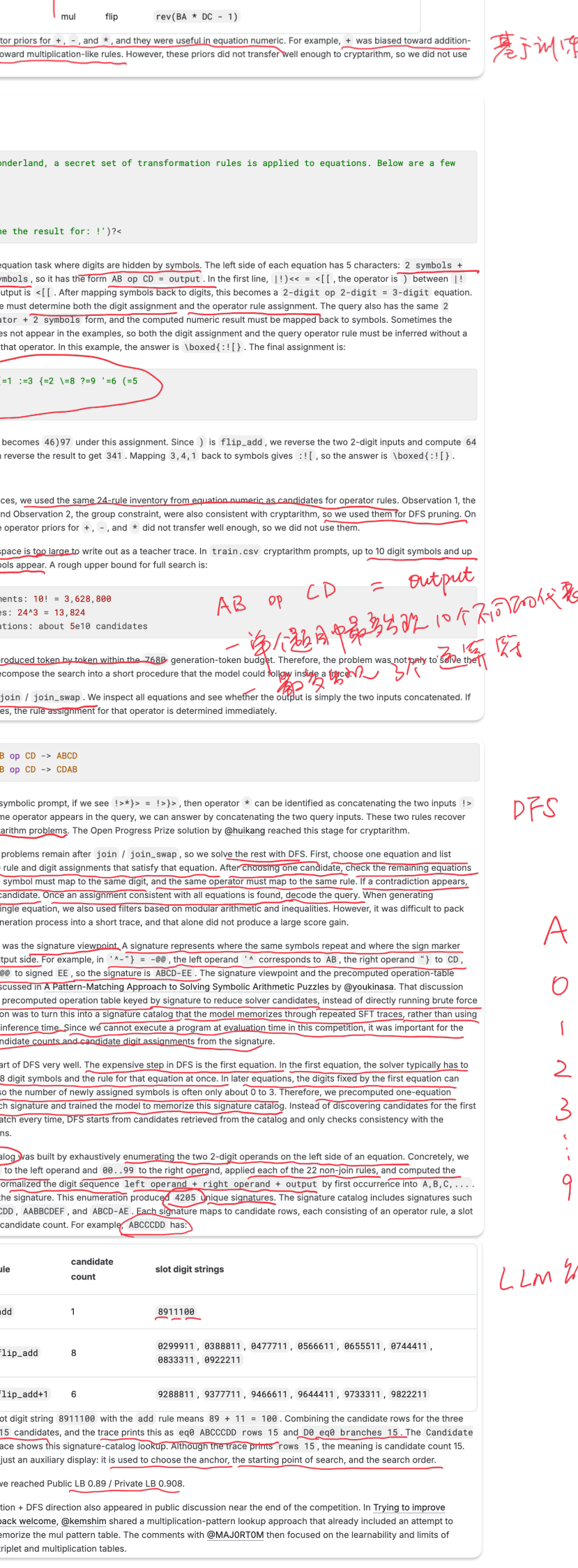


The 24-rule inventory is listed below. We write it using the 2-digit inputs in `AB op CD`. `BA` and `DC` are the reversed digits of `AB` and `CD`. `rev(x)` reverses the decimal representation of the result. `op` means that the operator symbol in the prompt is used as the sign marker.

rule	group	mode	definition
`join`	join	join	ABCD
`join_swap`	join	join	CDAB
`add`	add	normal	`AB + CD`
`add+1`	add	normal	`AB + CD + 1`
`add-1`	add	normal	`AB + CD - 1`
`flip_add`	add	flip	`rev(BA + DC)`
`flip_add+1`	add	flip	`rev(BA + DC + 1)`
`flip_add-1`	add	flip	`rev(BA + DC - 1)`
`sub`	sub	normal	`AB - CD`
`absdiff`	sub	normal	`abs(AB - CD)`
`absdiff_prefix`	sub	normal	`op + abs(AB - CD)` when `abs(AB - CD)` is nonzero
`maxmodmin`	sub	normal	`max(AB, CD) mod min(AB, CD)`
`flip_sub_prefix`	sub	flip	`rev(BA - DC)`, with `op` prefix if negative
`flip_sub_suffix`	sub	flip	`rev(DC - BA)`, with `op` suffix if negative
`flip_absdiff`	sub	flip	`rev(abs(BA - DC))`
`flip_absdiff_prefix`	sub	flip	`op + rev(abs(BA - DC))` when `abs(BA - DC)` is nonzero
`flip_absdiff_suffix`	sub	flip	`rev(abs(BA - DC)) + op` when `abs(BA - DC)` is nonzero
`flip_maxmodmin`	sub	flip	`rev(max(BA, DC) mod min(BA, DC))`
`mul`	mul	normal	`AB * CD`
`mul+1`	mul	normal	`AB * CD + 1`
`mul-1`	mul	normal	`AB * CD - 1`
`flip_mul`	mul	flip	`rev(BA * DC)`
`flip_mul+1`	mul	flip	`rev(BA * DC + 1)`
`flip_mul-1`	mul	flip	`rev(BA * DC - 1)`

There were operator priors for `+`, `-`, and `*`, and they were useful in equation numeric. For example, `+` was biased toward addition-like rules and `*` toward multiplication-like rules. However, these priors did not transfer well enough to cryptarithm, so we did not use them there.

基于训练数据统计



Cryptarithm

Task Format

```
In Alice's Wonderland, a secret set of transformation rules is applied to equations. Below are a few
examples:
|!)<< = <[[
::$\( = !(?
<<$'' = {\|(
Now, determine the result for: !')?<
```

Cryptarithm is an equation task where digits are hidden by symbols. The left side of each equation has 5 characters: `2 symbols + operator + 2 symbols`, so it has the form `AB op CD = output`. In the first line, `|!)<< = <[[`, the operator is `)` between `|!` and `<<`, and the output is `<[[`. After mapping symbols back to digits, this becomes a `2-digit op 2-digit = 3-digit` equation. In this category, we must determine both the digit assignment and the operator rule assignment. The query also has the same `2 symbols + operator + 2 symbols` form, and the computed numeric result must be mapped back to symbols. Sometimes the query operator does not appear in the examples, so both the digit assignment and the query operator rule must be inferred without a direct example for that operator. In this example, the answer is `\boxed{:![}`. The final assignment is:

```
|=0 !=4 <=7 [=1 :=3 {=2 \=8 ?=9 '=6 (=5
)=flip_add
$=flip_mul
```

The query `!')?<` becomes `46)97` under this assignment. Since `)` is `flip_add`, we reverse the two 2-digit inputs and compute `64 + 79 = 143`, then reverse the result to get `341`. Mapping `3,4,1` back to symbols gives `:![`, so the answer is `\boxed{:![}`.

Solution

For cryptarithm traces, we used the same 24-rule inventory from equation numeric as candidates for operator rules. Observation 1, the mode constraint, and Observation 2, the group constraint, were also consistent with cryptarithm, so we used them for DFS pruning. On the other hand, the operator priors for `+`, `-`, and `*` did not transfer well enough, so we did not use them.

The naive search space is too large to write out as a teacher trace. In `train.csv` cryptarithm prompts, up to 10 digit symbols and up to 3 operator symbols appear. A rough upper bound for full search is:

```
digit assignments: 10! = 3,628,800
operator rules: 24^3 = 13,824
total combinations: about 5e10 candidates
```

AB op CD = output

- 单个题目中最多出现 10 个不同的代表数字的符号

最多出现 3 个运算符

This cannot be reproduced token by token within the `7680` generation-token budget. Therefore, the problem was not only to solve the task, but also to decompose the search into a short procedure that the model could follow inside a trace.

The first check is `join` / `join_swap`. We inspect all equations and see whether the output is simply the two inputs concatenated. If an equation matches, the rule assignment for that operator is determined immediately.

```
join:      AB op CD -> ABCD
join_swap: AB op CD -> CDAB
```

For example, in a symbolic prompt, if we see `!>*}> = !>}>`, then operator `*` can be identified as concatenating the two inputs `!>` and `}>`. If the same operator appears in the query, we can answer by concatenating the two query inputs. These two rules recover about 5% of cryptarithm problems. The Open Progress Prize solution by @huikang reached this stage for cryptarithm.

Many cryptarithm problems remain after `join` / `join_swap`, so we solve the rest with DFS. First, choose one equation and list candidates for the rule and digit assignments that satisfy that equation. After choosing one candidate, check the remaining equations in order: the same symbol must map to the same digit, and the same operator must map to the same rule. If a contradiction appears, move to the next candidate. Once an assignment consistent with all equations is found, decode the query. When generating candidates for a single equation, we also used filters based on modular arithmetic and inequalities. However, it was difficult to pack that candidate-generation process into a short trace, and that alone did not produce a large score gain.

The breakthrough was the signature viewpoint. A signature represents where the same symbols repeat and where the sign marker appears on the output side. For example, in `'^-"} = -@@`, the left operand `'^` corresponds to `AB`, the right operand `"}` to `CD`, and the answer `-@@` to signed `EE`, so the signature is `ABCD-EE`. The signature viewpoint and the precomputed operation-table approach were discussed in A Pattern-Matching Approach to Solving Symbolic Arithmetic Puzzles by @youkinasa. That discussion described using a precomputed operation table keyed by signature to reduce solver candidates, instead of directly running brute force search. Our addition was to turn this into a signature catalog that the model memorizes through repeated SFT traces, rather than using it as a program at inference time. Since we cannot execute a program at evaluation time in this competition, it was important for the model to recall candidate counts and candidate digit assignments from the signature.

This fit the hard part of DFS very well. The expensive step in DFS is the first equation. In the first equation, the solver typically has to assign about 4 to 8 digit symbols and the rule for that equation at once. In later equations, the digits fixed by the first equation can already be used, so the number of newly assigned symbols is often only about 0 to 3. Therefore, we precomputed one-equation candidates for each signature and trained the model to memorize this signature catalog. Instead of discovering candidates for the first equation from scratch every time, DFS starts from candidates retrieved from the catalog and only checks consistency with the remaining equations.

The signature catalog was built by exhaustively enumerating the two 2-digit operands on the left side of an equation. Concretely, we assigned `00..99` to the left operand and `00..99` to the right operand, applied each of the 22 non-join rules, and computed the output. Then we normalized the digit sequence `left operand + right operand + output` by first occurrence into `A,B,C,...`. This sequence is the signature. This enumeration produced `4205` unique signatures. The signature catalog includes signatures such as `AAAAA`, `ABCCCDD`, `AABBCDEF`, and `ABCD-AE`. Each signature maps to candidate rows, each consisting of an operator rule, a slot digit string, and a candidate count. For example, `ABCCCDD` has:

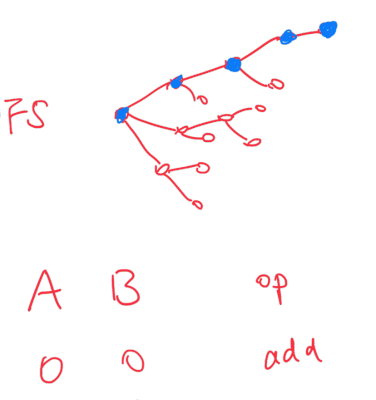
signature	rule	candidate count	slot digit strings
`ABCCCDD`	`add`	1	`8911100`
`ABCCCDD`	`flip_add`	8	`0299911`, `0388811`, `0477711`, `0566611`, `0655511`, `0744411`, `0833311`, `0922211`
`ABCCCDD`	`flip_add+1`	6	`9288811`, `9377711`, `9466611`, `9644411`, `9733311`, `9822211`

For example, the slot digit string `8911100` with the `add` rule means `89 + 11 = 100`. Combining the candidate rows for the three rules above gives `15` candidates, and the trace prints this as `eq0 ABCCCDD rows 15` and `D0 eq0 branches 15`. The `Candidate rows` part of the trace shows this signature-catalog lookup. Although the trace prints `rows 15`, the meaning is candidate count 15. This number is not just an auxiliary display: it is used to choose the anchor, the starting point of search, and the search order.

With this method, we reached Public LB 0.89 / Private LB 0.908.

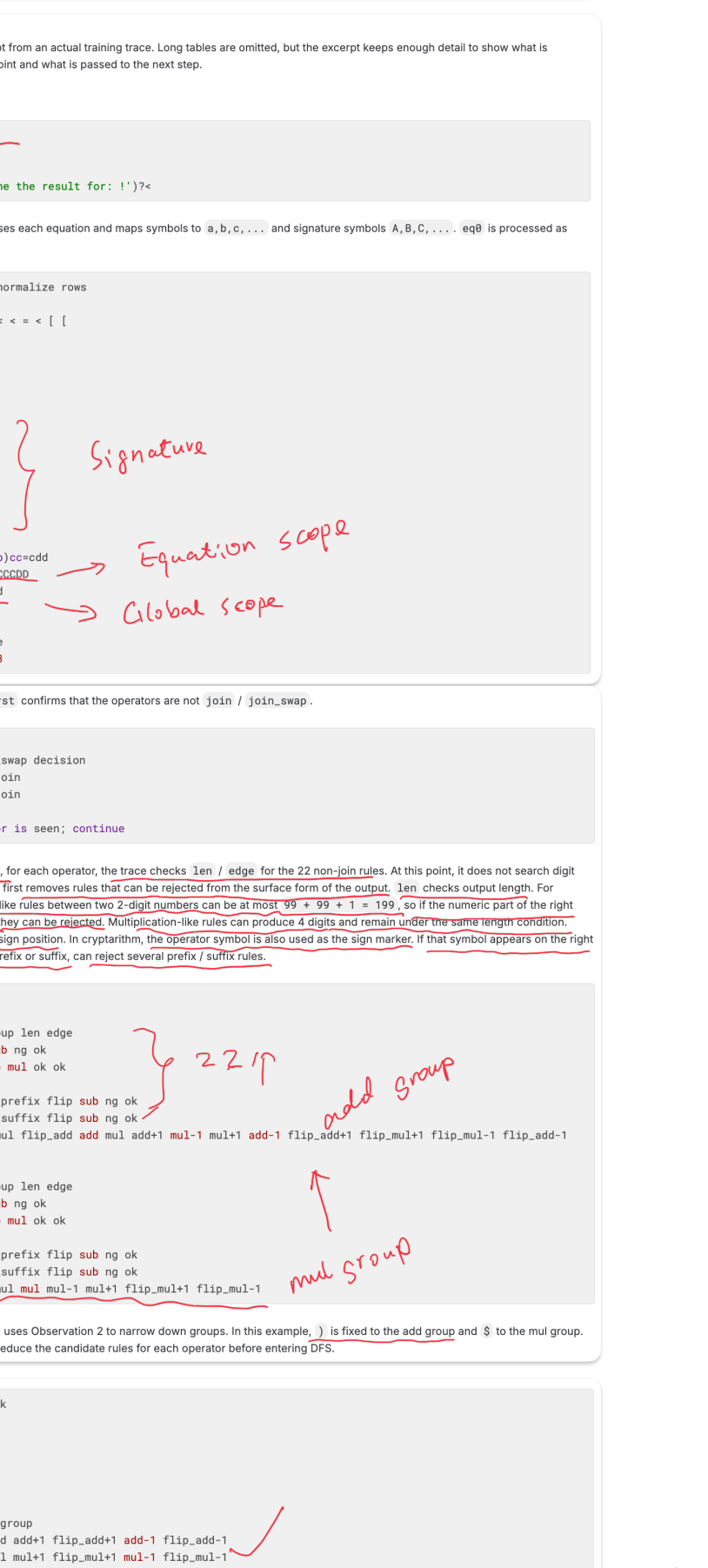
A similar memorization + DFS direction also appeared in public discussion near the end of the competition. In Trying to improve cryptarithm - feedback welcome, @kamshim shared a multiplication-pattern lookup approach that already included an attempt to make the model memorize the mul pattern table. The comments with @MAJORTOM then focused on the learnability and limits of memorizing those triplet and multiplication tables.

DFS



A	B	op	C	D	=	output	
0	0	add	0	0		0	0
1	1	mul	1	1		1	1
2	2	add+1	2	2		2	2
3	3	⋮	⋮	⋮		⋮	⋮
⋮	⋮						
9	9		9	9		9	9

LLM 能力边界, 可以记住大量的事实/执行步骤



Trace Example

Below is an excerpt from an actual training trace. Long tables are omitted, but the excerpt keeps enough detail to show what is decided at each point and what is passed to the next step.

Target prompt:

```
|!)<< = <[[
::$\( = !(?
<<$'' = {\|(
Now, determine the result for: !')?<
```

The trace first parses each equation and maps symbols to `a,b,c,...` and signature symbols `A,B,C,...`. `eq0` is processed as follows.

```
1 Parse and normalize rows
eq0
split | ! ) < < = < [ [
left | !
op )
right < <
output < [ [
map
| a A new
! b B new
) ) op
< c C new
< c C seen
< c C seen
[ d D new
[ d D seen
normalized ab)cc=cdd
signature ABCCCDD
slots abcccdd
join no
join_swap no
rhs_edge none
rhs_mag_len 3
```

Signature

Equation scope

Global scope

Next, `2 Join-first` confirms that the operators are not `join` / `join_swap`.

```
2 Join-first
op join join_swap decision
$ no no non_join
) no no non_join
query op )
query operator is seen; continue
```

In `3 Rule table`, for each operator, the trace checks `len` / `edge` for the 22 non-join rules. At this point, it does not search digit assignments yet. It first removes rules that can be rejected from the surface form of the output. `len` checks output length. For example, addition-like rules between two 2-digit numbers can be at most `99 + 99 + 1 = 199`, so if the numeric part of the right side has length 4, they can be rejected. Multiplication-like rules can produce 4 digits and remain under the same length condition. `edge` checks the sign position. In cryptarithm, the operator symbol is also used as the sign marker. If that symbol appears on the right side, its position, prefix or suffix, can reject several prefix / suffix rules.

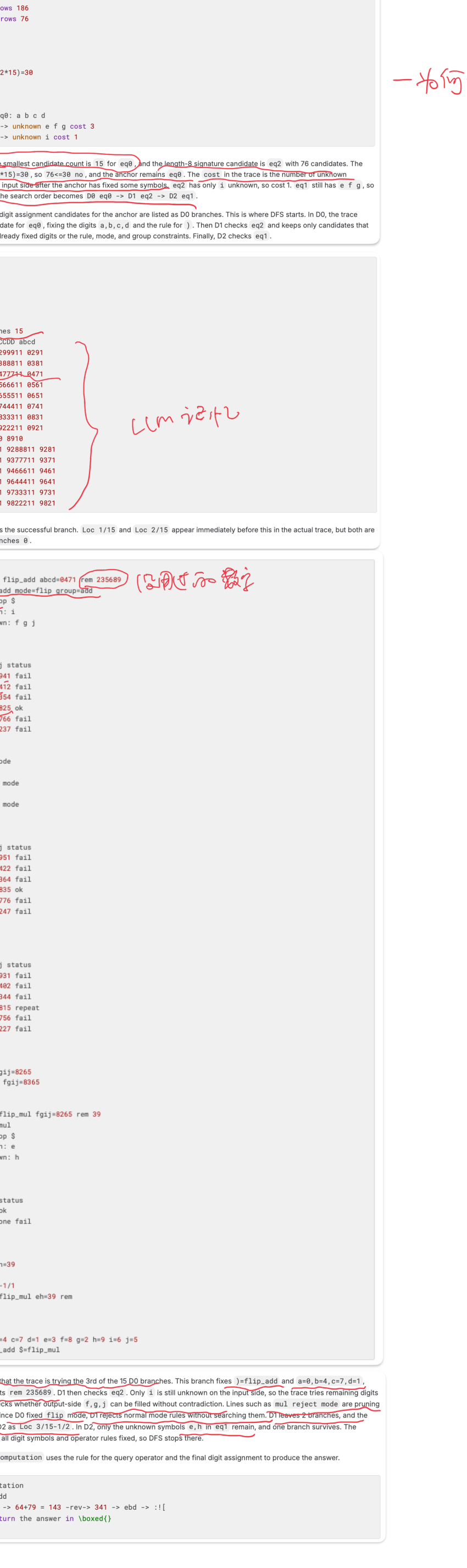
```
3 Rule table
op )
rule mode group len edge
sub normal sub ng ok
flip_mul flip mul ok ok
...
flip_absdiff_prefix flip sub ng ok
flip_absdiff_suffix flip sub ng ok
active flip_mul flip_add add mul add+1 mul-1 mul+1 add-1 flip_add+1 flip_mul+1 flip_mul-1 flip_add-1

op $
rule mode group len edge
sub normal sub ng ok
flip_mul flip mul ok ok
...
flip_absdiff_prefix flip sub ng ok
flip_absdiff_suffix flip sub ng ok
active flip_mul mul mul-1 mul+1 flip_mul+1 flip_mul-1
```

22个

add group

mul group

`Group precheck` uses Observation 2 to narrow down groups. In this example, `)` is fixed to the add group and `$` to the mul group. The purpose is to reduce the candidate rules for each operator before entering DFS.



```
Group precheck
op groups
) add mul
$ mul
match
) add
$ mul
active after group
) add flip_add add+1 flip_add+1 add-1 flip_add-1
$ mul flip_mul mul+1 flip_mul+1 mul-1 flip_mul-1
```

In `4 Candidate rows`, the trace retrieves the candidate count from the signature. This candidate count is also something the model memorizes. The anchor is first chosen from equations with the smallest candidate count. In addition, the trace also considers a length-8 signature where multiplication can be fixed early. In the excerpt this is printed as `sig8`. If the length-8 candidate count is at most `max(30, 2 * min_count)`, it is chosen as the anchor. Otherwise, the minimum-count equation remains the anchor.

```
4 Candidate rows
eq0 ABCCCDD rows 15
eq1 AABCDCE rows 186
eq2 AABBCDEF rows 76

anchor
min eq0 15
sig8 eq2 76
limit max(30,2*15)=30
76<=30 no
anchor eq0
order check
fixed after eq0: a b c d
eq1 lhs eefg -> unknown e f g cost 3
eq2 lhs ccii -> unknown i cost 1
```

In this example, the smallest candidate count is `15` for `eq0`, and the length-8 signature candidate is `eq2` with 76 candidates. The limit is `max(30, 2*15)=30`, so `76<=30 no`, and the anchor remains `eq0`. The `cost` in the trace is the number of unknown symbols left on the input side after the anchor has fixed some symbols. `eq2` has only `i` unknown, so cost 1. `eq1` still has `e f g`, so cost 3. Therefore, the search order becomes `D0 eq0 -> D1 eq2 -> D2 eq1`.

In `5 Search`, the digit assignment candidates for the anchor are listed as D0 branches. This is where DFS starts. In D0, the trace chooses one candidate for `eq0`, fixing the digits `a,b,c,d` and the rule for `)`. Then D1 checks `eq2` and keeps only candidates that do not contradict already fixed digits or the rule, mode, and group constraints. Finally, D2 checks `eq1`.

```
5 Search
order
D0 eq0
D1 eq2
D2 eq1

D0 eq0 branches 15
idx rule ABCCCDD abcd
1 flip_add 0299911 0291
2 flip_add 0388811 0381
3 flip_add 0477711 0471
4 flip_add 0566611 0561
5 flip_add 0655511 0651
6 flip_add 0744411 0741
7 flip_add 0833311 0831
8 flip_add 0922211 0921
9 add 8911100 8910
10 flip_add+1 9288811 9281
11 flip_add+1 9377711 9371
12 flip_add+1 9466611 9461
13 flip_add+1 9644411 9641
14 flip_add+1 9733311 9731
15 flip_add+1 9822211 9821
```

LLM 记忆化

The next excerpt is the successful branch. `Loc 1/15` and `Loc 2/15` appear immediately before this in the actual trace, but both are rejected with `branches 0`.

```
Loc 3/15
from D0 3/15 flip_add abcd=0471 rem 235689
lock )=flip_add mode=flip group=add
scan D1 eq2 op $
input unknown: i
output unknown: f g j

flip_mul
77*ii=j0fg
i lhs rhs fgj status
2 1694 j0fg 941 fail
3 2541 j0fg 412 fail
5 4235 j0fg 554 fail
6 5082 j0fg 825 ok
8 6776 j0fg 766 fail
9 7623 j0fg 237 fail
valid 1

mul reject mode

mul-1 reject mode

mul+1 reject mode

flip_mul+1
77*ii+1=j0fg
i lhs rhs fgj status
2 1695 j0fg 951 fail
3 2542 j0fg 422 fail
5 4236 j0fg 364 fail
6 5083 j0fg 835 ok
8 6777 j0fg 776 fail
9 7624 j0fg 247 fail
valid 1

flip_mul-1
77*ii-1=j0fg
i lhs rhs fgj status
2 1693 j0fg 931 fail
3 2540 j0fg 402 fail
5 4234 j0fg 344 fail
6 5081 j0fg 815 repeat
8 6775 j0fg 756 fail
9 7622 j0fg 227 fail
valid 0

branches 2
1 flip_mul fgij=8265
2 flip_mul+1 fgij=8365

Loc 3/15-1/2
from D1 1/2 flip_mul fgij=8265 rem 39
lock $=flip_mul
scan D2 eq1 op $
input unknown: e
output unknown: h

flip_mul
ee*28=h24
e lhs rhs h status
3 924 h24 9 ok
9 2772 h24 none fail
valid 1

branches 1
1 flip_mul eh=39

Loc 3/15-1/2-1/1
from D2 1/1 flip_mul eh=39 rem

final state
path 3.1.1
digits a=0 b=4 c=7 d=1 e=3 f=8 g=2 h=9 i=6 j=5
rules )=flip_add $=flip_mul
```

没用过的数字

`Loc 3/15` means that the trace is trying the 3rd of the 15 D0 branches. This branch fixes `)=flip_add` and `a=0,b=4,c=7,d=1`, leaving unused digits `rem 235689`. D1 then checks `eq2`. Only `i` is still unknown on the input side, so the trace tries remaining digits one by one and checks whether output-side `f,g,j` can be filled without contradiction. Lines such as `mul reject mode` are pruning by Observation 1. Since D0 fixed `flip` mode, D1 rejects normal mode rules without searching them. D1 leaves 2 branches, and the first one moves to D2 as `Loc 3/15-1/2`. In D2, only the unknown symbols `e,h` in `eq1` remain, and one branch survives. The `final state` has all digit symbols and operator rules fixed, so DFS stops there.

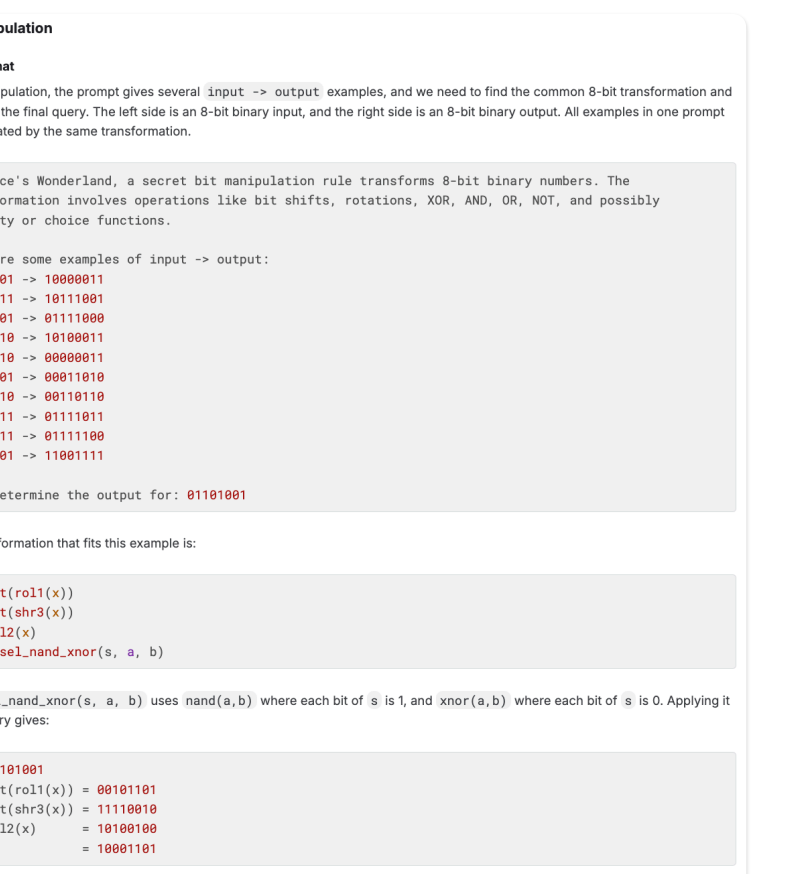
Finally, `6 Query computation` uses the rule for the query operator and the final digit assignment to produce the answer.

```
6 Query computation
rule ) flip_add
bi)hc = 46)97 -> 64+79 = 143 -rev-> 341 -> ebd -> :![
I will now return the answer in \boxed{}
\boxed{:![}
```

常规策略：

- 启发式：如果方程 signature 为8，暗示 op 更可能为乘法，可固定更多的符号

- 为何 30 阈值？ max(30, 2× min_count)



Bit Manipulation

Task Format

In bit manipulation, the prompt gives several `input -> output` examples, and we need to find the common 8-bit transformation and apply it to the final query. The left side is an 8-bit binary input, and the right side is an 8-bit binary output. All examples in one prompt are generated by the same transformation.

```
In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary numbers. The
transformation involves operations like bit shifts, rotations, XOR, AND, OR, NOT, and possibly
majority or choice functions.

Here are some examples of input -> output:
00010101 -> 10000011
01100011 -> 10111001
11000101 -> 01111000
00011010 -> 10100011
01010110 -> 00000011
11010001 -> 00011010
10100010 -> 00110110
00111111 -> 01111011
11000011 -> 01111100
01110101 -> 11001111

Now, determine the output for: 01101001
```

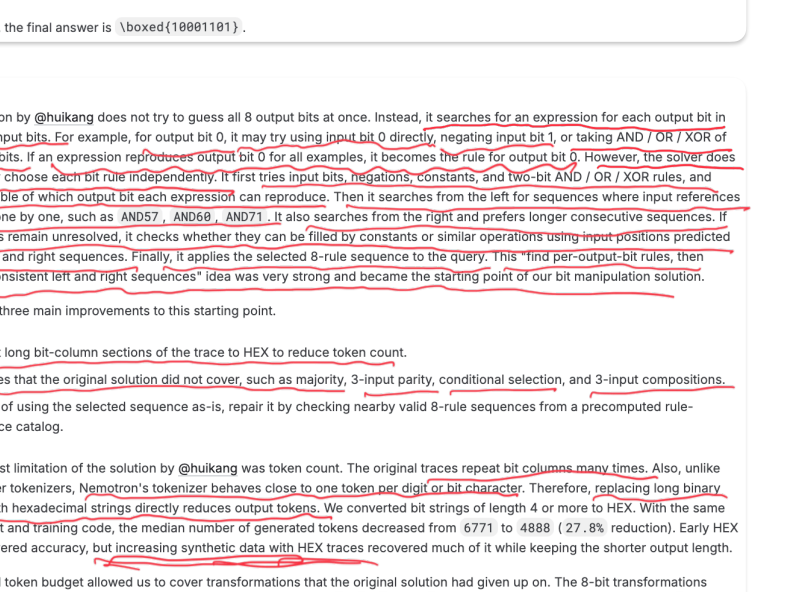
One transformation that fits this example is:

```
s = not(rol1(x))
a = not(shr3(x))
b = shl2(x)
out = sel_nand_xnor(s, a, b)
```

Here, `sel_nand_xnor(s, a, b)` uses `nand(a,b)` where each bit of `s` is 1, and `xnor(a,b)` where each bit of `s` is 0. Applying it to the query gives:

```
x = 01101001
s = not(rol1(x)) = 00101101
a = not(shr3(x)) = 11110010
b = shl2(x)      = 10100100
out              = 10001101
```

Therefore, the final answer is `\boxed{10001101}`.



Solution

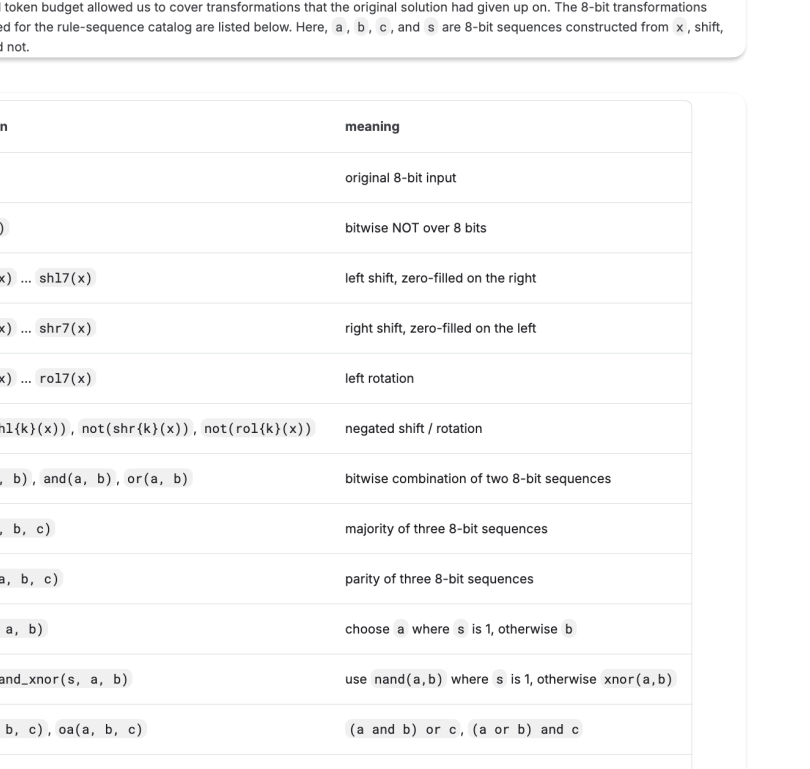
The solution by @huikang does not try to guess all 8 output bits at once. Instead, it searches for an expression for each output bit in terms of input bits. For example, for output bit 0, it may try using input bit 0 directly, negating input bit 1, or taking AND / OR / XOR of two input bits. If an expression reproduces output bit 0 for all examples, it becomes the rule for output bit 0. However, the solver does not simply choose each bit rule independently. It first tries input bits, negations, constants, and two-bit AND / OR / XOR rules, and builds a table of which output bit each expression can reproduce. Then it searches from the left for sequences where input references advance one by one, such as `AND57`, `AND60`, `AND71`. It also searches from the right and prefers longer consecutive sequences. If middle bits remain unresolved, it checks whether they can be filled by constants or similar operations using input positions predicted by the left and right sequences. Finally, it applies the selected 8-rule sequence to the query. This "find per-output-bit rules, then choose consistent left and right sequences" idea was very strong and became the starting point of our bit manipulation solution.

We made three main improvements to this starting point.

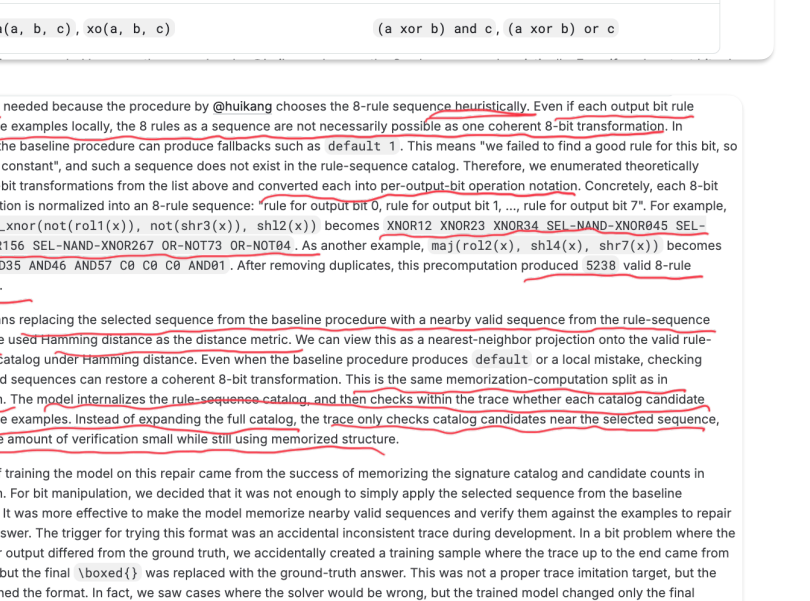
1. Convert long bit-column sections of the trace to HEX to reduce token count.
2. Add rules that the original solution did not cover, such as majority, 3-input parity, conditional selection, and 3-input compositions.
3. Instead of using the selected sequence as-is, repair it by checking nearby valid 8-rule sequences from a precomputed rule-sequence catalog.

The biggest limitation of the solution by @huikang was token count. The original traces repeat bit columns many times. Also, unlike many other tokenizers, Nemotron's tokenizer behaves close to one token per digit or bit character. Therefore, replacing long binary strings with hexadecimal strings directly reduces output tokens. We converted bit strings of length 4 or more to HEX. With the same sample set and training code, the median number of generated tokens decreased from `6771` to `4888` (`27.8%` reduction). Early HEX traces lowered accuracy, but increasing synthetic data with HEX traces recovered much of it while keeping the shorter output length.

The saved token budget allowed us to cover transformations that the original solution had given up on. The 8-bit transformations enumerated for the rule-sequence catalog are listed below. Here, `a`, `b`, `c`, and `s` are 8-bit sequences constructed from `x`, shift, rotate, and not.



notation	meaning
`x`	original 8-bit input
`not(x)`	bitwise NOT over 8 bits
`shl1(x)` ... `shl7(x)`	left shift, zero-filled on the right
`shr1(x)` ... `shr7(x)`	right shift, zero-filled on the left
`rol1(x)` ... `rol7(x)`	left rotation
`not(shl{k}(x))`, `not(shr{k}(x))`, `not(rol{k}(x))`	negated shift / rotation
`xor(a, b)`, `and(a, b)`, `or(a, b)`	bitwise combination of two 8-bit sequences
`maj(a, b, c)`	majority of three 8-bit sequences
`par3(a, b, c)`	parity of three 8-bit sequences
`ch(s, a, b)`	choose `a` where `s` is 1, otherwise `b`
`sel_nand_xnor(s, a, b)`	use `nand(a,b)` where `s` is 1, otherwise `xnor(a,b)`
`ao(a, b, c)`, `oa(a, b, c)`	`(a and b) or c`, `(a or b) and c`
`ax(a, b, c)`, `ox(a, b, c)`	`(a and b) xor c`, `(a or b) xor c`
`xa(a, b, c)`, `xo(a, b, c)`	`(a xor b) and c`, `(a xor b) or c`

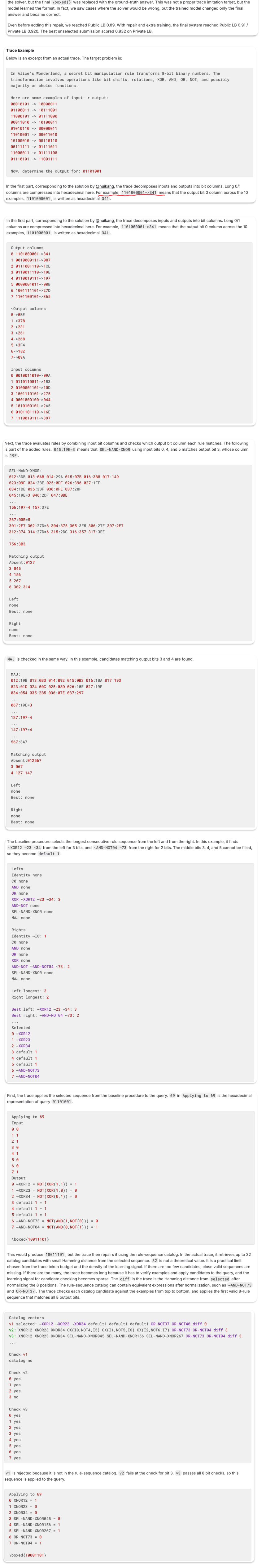


Repair was needed because the procedure by @huikang chooses the 8-rule sequence heuristically. Even if each output bit rule matches the examples locally, the 8 rules as a sequence are not necessarily possible as one coherent 8-bit transformation. In particular, the baseline procedure can produce fallbacks such as `default 1`. This means "we failed to find a good rule for this bit, so fill it with a constant", and such a sequence does not exist in the rule-sequence catalog. Therefore, we enumerated theoretically possible 8-bit transformations from the list above and converted each into per-output-bit operation notation. Concretely, each 8-bit transformation is normalized into an 8-rule sequence: "rule for output bit 0, rule for output bit 1, ..., rule for output bit 7". For example, `sel_nand_xnor(not(rol1(x)), not(shr3(x)), shl2(x))` becomes `XNOR12 XNOR23 XNOR34 SEL-NAND-XNOR045 SEL-NAND-XNOR156 SEL-NAND-XNOR267 OR-NOT73 OR-NOT04`. As another example, `maj(rol2(x), shl4(x), shr7(x))` becomes `AND24 AND35 AND46 AND57 C0 C0 C0 AND01`. After removing duplicates, this precomputation produced `5238` valid 8-rule sequences.

Repair means replacing the selected sequence from the baseline procedure with a nearby valid sequence from the rule-sequence catalog. We used Hamming distance as the distance metric. We can view this as a nearest-neighbor projection onto the valid rule-sequence catalog under Hamming distance. Even when the baseline procedure produces `default` or a local mistake, checking nearby valid sequences can restore a coherent 8-bit transformation. This is the same memorization-computation split as in cryptarithm. The model internalizes the rule-sequence catalog, and then checks within the trace whether each catalog candidate matches the examples. Instead of expanding the full catalog, the trace only checks catalog candidates near the selected sequence, keeping the amount of verification small while still using memorized structure.

The idea of training the model on this repair came from the success of memorizing the signature catalog and candidate counts in cryptarithm. For bit manipulation, we decided that it was not enough to simply apply the selected sequence from the baseline procedure. It was more effective to make the model memorize nearby valid sequences and verify them against the examples to repair the final answer. The trigger for trying this format was an accidental inconsistent trace during development. In a bit problem where the local solver output differed from the ground truth, we accidentally created a training sample where the trace up to the end came from the solver, but the final `\boxed{}` was replaced with the ground-truth answer. This was not a proper trace imitation target, but the model learned the format. In fact, we saw cases where the solver would be wrong, but the trained model changed only the final answer and became correct.

Even before adding this repair, we reached Public LB 0.89. With repair and extra training, the final system reached Public LB 0.91 / Private LB 0.920. The best unselected submission scored 0.932 on Private LB.



Trace Example

Below is an excerpt from an actual trace. The target problem is:

```
In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary numbers. The
transformation involves operations like bit shifts, rotations, XOR, AND, OR, NOT, and possibly
majority or choice functions.

Here are some examples of input -> output:
00010101 -> 10000011
01100011 -> 10111001
11000101 -> 01111000
00011010 -> 10100011
01010110 -> 00000011
11010001 -> 00011010
10100010 -> 00110110
00111111 -> 01111011
11000011 -> 01111100
01110101 -> 11001111

Now, determine the output for: 01101001
```

In the first part, corresponding to the solution by @huikang, the trace decomposes inputs and outputs into bit columns. Long 0/1 columns are compressed into hexadecimal here. For example, `1101000001->341` means that the output bit 0 column across the 10 examples, `1101000001`, is written as hexadecimal `341`.

In the first part, corresponding to the solution by @huikang, the trace decomposes inputs and outputs into bit columns. Long 0/1 columns are compressed into hexadecimal here. For example, `1101000001->341` means that the output bit 0 column across the 10 examples, `1101000001`, is written as hexadecimal `341`.

```
Output columns
0 1101000001->341
1 0010000111->087
2 0111001110->1CE
3 0110011110->19E
4 0110010111->197
5 0000001011->00B
6 1001111101->27D
7 1101100101->365

~Output columns
0->0BE
1->378
2->231
3->261
4->268
5->3F4
6->182
7->09A

Input columns
0 0010011010->09A
1 0110110011->1B3
2 0100001101->10D
3 1001110101->275
4 0001000100->044
5 1010100101->2A5
6 0101101110->16E
7 1110010111->397
```

Next, the trace evaluates rules by combining input bit columns and checks which output bit column each rule matches. The following is part of the added rules. `045:19E=3` means that `SEL-NAND-XNOR` using input bits 0, 4, and 5 matches output bit 3, whose column is `19E`.

```
SEL-NAND-XNOR:
012:3DB 013:0AB 014:29A 015:07B 016:3B8 017:149
023:09F 024:2BE 025:0DF 026:396 027:1FF
034:1DE 035:3BF 036:0FE 037:28F
045:19E=3 046:2DF 047:0BE
...
156:197=4 157:37E
...
267:00B=5
301:2E7 302:27D=6 304:375 305:3F5 306:27F 307:2E7
312:374 314:27D=6 315:2DC 316:357 317:3EE
...
756:3B3

Matching output
Absent:0127
3 045
4 156
5 267
6 302 314

Left
none
Best: none

Right
none
Best: none
```

`MAJ` is checked in the same way. In this example, candidates matching output bits 3 and 4 are found.

```
MAJ:
012:19B 013:0B3 014:092 015:0B3 016:1BA 017:193
023:01D 024:00C 025:08D 026:10E 027:19F
034:054 035:2B5 036:07E 037:297
...
067:19E=3
...
127:197=4
...
147:197=4
...
567:3A7

Matching output
Absent:012567
3 067
4 127 147

Left
none
Best: none

Right
none
Best: none
```

The baseline procedure selects the longest consecutive rule sequence from the left and from the right. In this example, it finds `~XOR12 ~23 ~34` from the left for 3 bits, and `~AND-NOT04 ~73` from the right for 2 bits. The middle bits 3, 4, and 5 cannot be filled, so they become `default 1`.

```
Lefts
Identity none
C0 none
AND none
OR none
XOR ~XOR12 ~23 ~34: 3
AND-NOT none
SEL-NAND-XNOR none
MAJ none

Rights
Identity ~I0: 1
C0 none
AND none
OR none
XOR none
AND-NOT ~AND-NOT04 ~73: 2
SEL-NAND-XNOR none
MAJ none

Left longest: 3
Right longest: 2

Best left: ~XOR12 ~23 ~34: 3
Best right: ~AND-NOT04 ~73: 2
...
Selected
0 ~XOR12
1 ~XOR23
2 ~XOR34
3 default 1
4 default 1
5 default 1
6 ~AND-NOT73
7 ~AND-NOT04
```

First, the trace applies the selected sequence from the baseline procedure to the query. `69` in `Applying to 69` is the hexadecimal representation of query `01101001`.

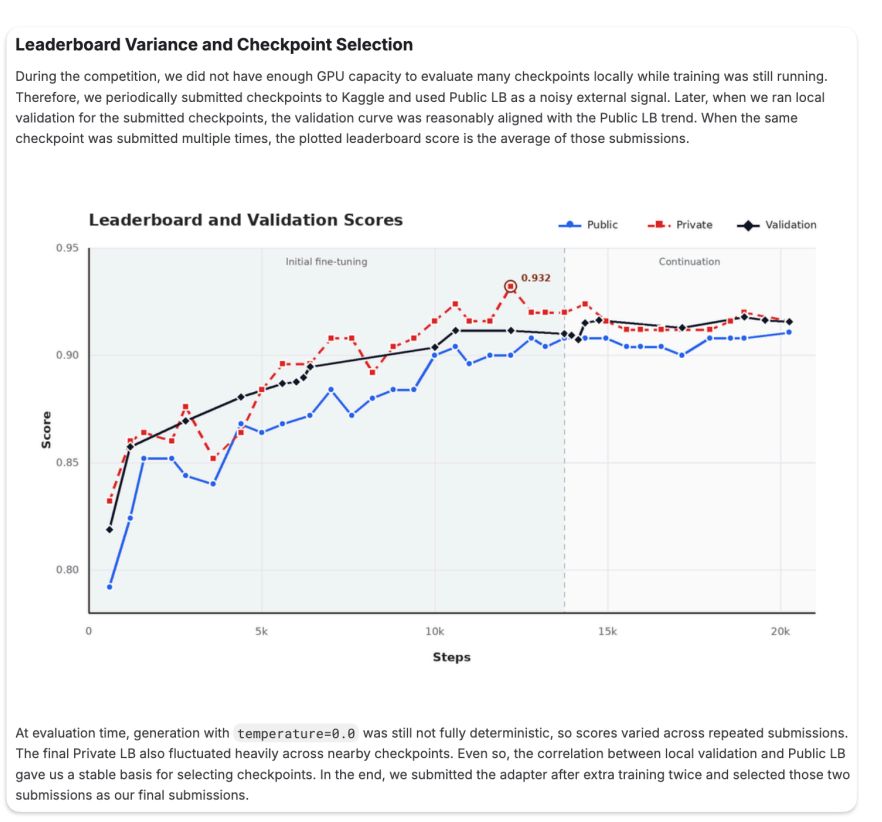
```
Applying to 69
Input
0 0
1 1
2 1
3 0
4 1
5 0
6 0
7 1
Output
0 ~XOR12 = NOT(XOR(1,1)) = 1
1 ~XOR23 = NOT(XOR(1,0)) = 0
2 ~XOR34 = NOT(XOR(0,1)) = 0
3 default 1 = 1
4 default 1 = 1
5 default 1 = 1
6 ~AND-NOT73 = NOT(AND(1,NOT(0))) = 0
7 ~AND-NOT04 = NOT(AND(0,NOT(1))) = 1

\boxed(10011101)
```

This would produce `10011101`, but the trace then repairs it using the rule-sequence catalog. In the actual trace, it retrieves up to 32 catalog candidates with small Hamming distance from the selected sequence. `32` is not a theoretical value. It is a practical limit chosen from the trace token budget and the density of the learning signal. If there are too few candidates, close valid sequences are missing. If there are too many, the trace becomes long because it has to verify examples and apply candidates to the query, and the learning signal for candidate checking becomes sparse. The `diff` in the trace is the Hamming distance from `selected` after normalizing the 8 positions. The rule-sequence catalog can contain equivalent expressions after normalization, such as `~AND-NOT73` and `OR-NOT37`. The trace checks each catalog candidate against the examples from top to bottom, and applies the first valid 8-rule sequence that matches all 8 output bits.

```
Catalog vectors
v1 selected: ~XOR12 ~XOR23 ~XOR34 default1 default1 default1 OR-NOT37 OR-NOT40 diff 0
v2: XNOR12 XNOR23 XNOR34 OX(I0,NOT4,I5) OX(I1,NOT5,I6) OX(I2,NOT6,I7) OR-NOT73 OR-NOT04 diff 3
v3: XNOR12 XNOR23 XNOR34 SEL-NAND-XNOR045 SEL-NAND-XNOR156 SEL-NAND-XNOR267 OR-NOT73 OR-NOT04 diff 3
...

Check v1
catalog no

Check v2
0 yes
1 yes
2 yes
3 no

Check v3
0 yes
1 yes
2 yes
3 yes
4 yes
5 yes
6 yes
7 yes
```

`v1` is rejected because it is not in the rule-sequence catalog. `v2` fails at the check for bit 3. `v3` passes all 8 bit checks, so this sequence is applied to the query.

```
Applying to 69
0 XNOR12 = 1
1 XNOR23 = 0
2 XNOR34 = 0
3 SEL-NAND-XNOR045 = 0
4 SEL-NAND-XNOR156 = 1
5 SEL-NAND-XNOR267 = 1
6 OR-NOT73 = 0
7 OR-NOT04 = 1

\boxed(10001101)
```

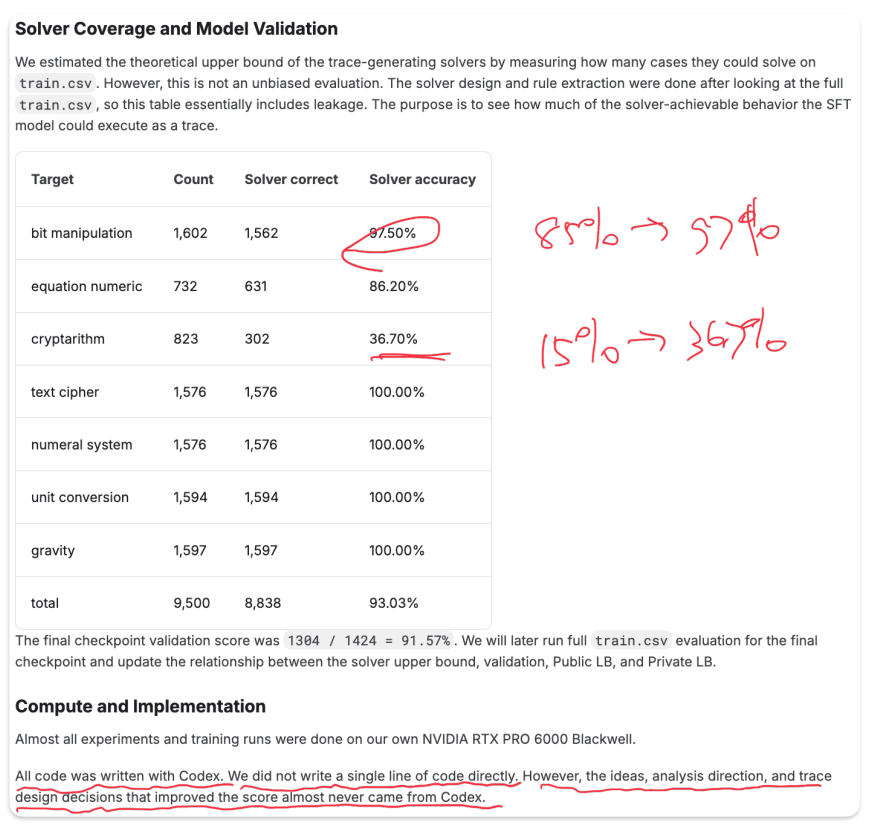


Leaderboard Variance and Checkpoint Selection

During the competition, we did not have enough GPU capacity to evaluate many checkpoints locally while training was still running. Therefore, we periodically submitted checkpoints to Kaggle and used Public LB as a noisy external signal. Later, when we ran local validation for the submitted checkpoints, the validation curve was reasonably aligned with the Public LB trend. When the same checkpoint was submitted multiple times, the plotted leaderboard score is the average of those submissions.

Leaderboard and Validation Scores

At evaluation time, generation with `temperature=0.0` was still not fully deterministic, so scores varied across repeated submissions. The final Private LB also fluctuated heavily across nearby checkpoints. Even so, the correlation between local validation and Public LB gave us a stable basis for selecting checkpoints. In the end, we submitted the adapter after extra training twice and selected those two submissions as our final submissions.



Solver Coverage and Model Validation

We estimated the theoretical upper bound of the trace-generating solvers by measuring how many cases they could solve on `train.csv`. However, this is not an unbiased evaluation. The solver design and rule extraction were done after looking at the full `train.csv`, so this table essentially includes leakage. The purpose is to see how much of the solver-achievable behavior the SFT model could execute as a trace.

Target	Count	Solver correct	Solver accuracy
bit manipulation	1,602	1,562	97.50%
equation numeric	732	631	86.20%
cryptarithm	823	302	36.70%
text cipher	1,576	1,576	100.00%
numeral system	1,576	1,576	100.00%
unit conversion	1,594	1,594	100.00%
gravity	1,597	1,597	100.00%
total	9,500	8,838	93.03%

85% → 97%

15% → 36%

The final checkpoint validation score was `1304 / 1424 = 91.57%`. We will later run full `train.csv` evaluation for the final checkpoint and update the relationship between solver support, model validation, Public LB, and Private LB.

Compute and Implementation

Almost all experiments and training runs were done on our own NVIDIA RTX PRO 6000 Blackwell.

All code was written with Codex. We did not write a single line of code directly. However, the ideas, analysis direction, and trace design decisions that improved the score almost never came from Codex.